

Rapport d'analyse de données en python

Imene Hidri
06/12/2025

Table des matières

Introduction.....	2
Chargement et exploration des données	3
Correction et nettoyage des données.....	4
Détections d'anomalies	6
Analyse des données et KPI.....	7
Datatelling : qui est la cible ?	10
Scenario d'attaque	10

Introduction

Ce rapport porte sur l'analyse d'un fichier de résultats de campagnes de phishing simulées (result.csv).

Chaque ligne représente une personne testée, avec ses centres d'intérêt (jeux vidéo, design Instagram, football), son âge, le canal utilisé pour la cibler (mail, Facebook, Instagram, etc.), le produit utilisé comme appât (FIFA, Fortnite, Instagram pack...) et l'issue de la campagne (campaign_success, succès ou échec).

L'objectif n'est pas de lancer une vraie attaque, mais de comprendre quels profils se font le plus piéger, quels types de messages fonctionnent le mieux et via quels canaux. À partir de ces observations, je dois proposer un scénario d'attaque ciblée réaliste, appuyé sur les données et sur des graphiques. Le jeu de données est entièrement synthétique et sert uniquement à des fins pédagogiques.

Dans ce rapport, je raconte cette démarche étape par étape : d'abord l'exploration et le nettoyage des données, ensuite la détection des anomalies, puis l'analyse statistique, et enfin la construction d'un scénario d'attaque ciblée cohérent avec la réflexion et l'analyse des données.

L'idée est de montrer comment mes questions, mes tests et mes doutes m'ont amené aux choix que je présente dans la suite.

Chargement et exploration des données

Pour commencer, j'ai chargé le fichier result.csv avec Python et la bibliothèque pandas, en utilisant le séparateur ; qui est utilisé dans ce fichier. J'ai ensuite affiché les premières lignes, les types de colonnes et le nombre de valeurs manquantes. Cette première étape m'a permis de voir rapidement la structure du jeu de données.

```
7 df = pd.read_csv("result.csv", sep=";")
8
9 print("Premieres lignes:")
10 print(df.head())
11
12 print("\nTypes de colonnes:")
13 print(df.dtypes)
14
15 print("\nValeurs manquantes:")
16 print(df.isna().sum())
17
18 print("Nb lignes:", df.shape[0])
19 print("Nb colonnes :", df.shape[1])
20
```

```
Premieres lignes:
  Id  gaming_interest_score  insta_design_interest_s
0  1                      51
1  2                      60
2  3                      74
3  4                       2
4  5                      29

Types de colonnes:
Id                      int64
gaming_interest_score   int64
insta_design_interest_score  int64
football_interest_score  float64
recommended_product      object
campaign_success          object
age                      float64
canal_recommande         object
dtype: object

Valeurs manquantes:
Id                      0
gaming_interest_score   0
insta_design_interest_score  0
football_interest_score  1
recommended_product      2
campaign_success          0
age                      3
canal_recommande         0
dtype: int64
Nb lignes: 519
Nb colonnes : 8
```

À ce stade, j'ai constaté que le fichier contenait 519 lignes et 8 colonnes :

- Un identifiant (Id)
- Trois scores d'intérêt
(gaming_interest_score, insta_design_interest_score, football_interest_score)
- L'âge (age)
- Le produit recommandé (recommended_product)
- Le canal utilisé (canal_recommande)
- Le succès de la campagne (campaign_success)

Les scores étaient au départ de type entier ou flottant, l'âge en flottant, et plusieurs colonnes importantes (recommended_product, campaign_success, canal_recommande) étaient en texte brut.

Une première chose qui m'a frappé est la présence de valeurs manquantes : une valeur manquante dans `football_interest_score`, deux dans `recommended_product` et trois dans `age`.

Première question que je me suis posée était ce que je supprime les lignes qui n'ont pas ces valeurs ou est-ce que j'essaie de les remplir et quel est l'impact que ces données ont. J'ai très rapidement répondu à cette question en pensant qu'il était mieux de supprimer les lignes de recommandations de produits et d'âge. Puisque j'avais l'impression que ces 2 champs avaient plus d'impact sur une étude future.

Correction et nettoyage des données

Après l'exploration, j'ai commencé à nettoyer le fichier pour le rendre exploitable.

La première étape a été de corriger les types de données. Les trois scores d'intérêt ont été convertis en nombres flottants (`float32`), et l'âge en numérique. Cela permet d'éviter les problèmes si certaines valeurs sont mal écrites.

```
1 # Corrections des types
2 df["gaming_interest_score"] = df["gaming_interest_score"].astype("float32")
3 df["insta_design_interest_score"] = df["insta_design_interest_score"].astype("float32")
4 df["football_interest_score"] = df["football_interest_score"].astype("float32")
5
6 df["age"] = pd.to_numeric(df["age"], errors="coerce")
7
```

La colonne `campaign_success`, a été nettoyée avec `str.strip()` puis convertie en booléen (`True / False`) à l'aide d'un mapping. L'idée était que chaque colonne ait un type cohérent avec ce qu'elle représente.

Ensuite, j'ai traité les valeurs manquantes. Juste avant, j'avais repéré plusieurs valeurs manquantes. J'ai commencé par supprimer les lignes qui m'intéressaient, Mais en affichant les colonnes qui avaient des valeurs manquantes j'ai remarqué que celle du `football_score` avait disparu alors que je ne l'avais pas supprimée au préalable.

J'ai donc affiché la ligne où le score de football était manquant et j'ai vu qu'elle avait aussi un `recommended_product` vide, c'est pour cela que elle avait été supprimée.

En supprimant ces quelques lignes, la taille du dataset est passée de 519 à 515 lignes, et il ne restait plus aucune valeur manquante ni dans `age`, ni dans `recommended_product`, ni dans `football_interest_score`.

J'ai ensuite nettoyé les colonnes textuelles pour éviter les doublons cachés. Par exemple, la colonne canal_recommande contenait des variantes en majuscule, en minuscule et ainsi de suite. J'ai donc tout passé en minuscules et en supprimant les espaces inutiles. J'ai fait la même chose pour recommended_product, ce qui m'a permis de regrouper correctement des catégories.

```
df["canal_recommande"] = df["canal_recommande"].astype(str).str.strip().str.lower()
df["recommended_product"] = df["recommended_product"].astype(str).str.strip().str.lower()

print("\nExemples de valeurs pour canal_recommande :")
print(df["canal_recommande"].value_counts().head())

print("\nExemples de valeurs pour recommended_product :")
print(df["recommended_product"].value_counts().head())
```

```
Exemples de valeurs pour canal_recommande :
canal_recommande
mail          211
insta         162
facebook      132
non_defini    10
Name: count, dtype: int64

Exemples de valeurs pour recommended_product :
recommended_product
fortnite       181
fifa           179
instagram pack  153
fornite         1
test            1
Name: count, dtype: int64
```

A ce moment-là j'ai vu apparaître des valeurs comme test dans les produits et non_defini dans les canaux. Au début, ça m'a surpris, puis j'ai décidé de les supprimer puisque ça ne représente pas de vrais appâts ou de vrais canaux de fishing et ce sont plutôt des cas de test ou incomplets.

À la fin de cette phase de nettoyage, j'avais un premier jeu de données propre de base et c'est à partir de ce dataframe que j'ai ensuite pu chercher les vraies anomalies de valeurs et construire un jeu de données encore plus propre pour l'analyse statistique.

Détections d'anomalies

Une fois le nettoyage de base terminé, j'ai cherché à identifier les valeurs aberrantes qui pouvaient fausser les analyses statistiques.

La première approche que j'ai eu, était de regarder les statistiques descriptives qui m'ont tout de suite montré qu'il y avait des problèmes au niveau des scores avec des valeurs des fois négative et parfois énormes. Comme ces colonnes représentent des taux d'intérêt pour le gaming, Instagram et le football, j'ai fait une hypothèse simple : ces scores devraient être compris entre 0 et 100, comme un pourcentage.

Plutôt que de partir sur une méthode compliquée, j'ai choisi une règle simple : toute valeur de score strictement inférieure à 0 ou strictement supérieure à 100 est considérée comme une anomalie, et donc va être supprimé.

Concrètement, j'ai créé un filtre sur les trois colonnes de scores et le résultat m'a donné 12 lignes anormales. En affichant quelques exemples, on retrouvait précisément les valeurs comme un gaming_interest_score à -100, d'autres à 245, 329, 430, 590 ou 740, et des scores de football ou d'Instagram qui montaient jusqu'à 654 ou 742.

```
print("\nStatistiques descriptives après premier nettoyage")
print(df.describe())

anomalies_scores = df[
    (df["gaming_interest_score"] < 0) | (df["gaming_interest_score"] > 100) |
    (df["insta_design_interest_score"] < 0) | (df["insta_design_interest_score"] > 100) |
    (df["football_interest_score"] < 0) | (df["football_interest_score"] > 100)
]

print("\nNb lignes avec score pas normal) :", len(anomalies_scores))
print("\nExemples:")
print(anomalies_scores.head())
```

À côté des anomalies de scores, j'ai aussi pris une décision sur la variable age. Dans le cadre de ces analyses, on doit se concentrer sur des personnes de 20 ans et plus, qui correspondent mieux à des utilisateurs autonomes.

J'ai donc filtré le dataset pour ne garder que les lignes où age >= 20. Plus qu'anomalie, j'ai fait un filtrage : je restreins volontairement mon analyse à une tranche d'âge pertinente pour le scénario d'attaque que je souhaite construire.

J'ai ainsi créé un nouveau dataset sans anomalies et nettoyé.

Analyse des données et KPI

Maintenant que toutes ces étapes ont été terminées j'ai finalement pu commencer à analyser les comportements de façon plus globale.

L'idée était de répondre à des questions comme : quel est le taux de réussite moyen des attaques ? Quels appâts fonctionnent le mieux ? Quelle tranche d'âge sont les plus vulnérables ?

Pour cela, j'ai calculé plusieurs indicateurs simples (KPI) à partir de `df_clean`.

Tout d'abord, j'ai mesuré le taux de réussite global des campagnes de phishing en calculant la moyenne de la colonne `campaign_success` (les valeurs `True` étant considérées comme 1 et `False` comme 0).

```
# KPI 1: Le taux de réussite globale
success_rate_global = df_clean["campaign_success"].mean()
taux_pourcent = round(success_rate_global * 100, 2)
print("Taux de réussite global de la campagne :", taux_pourcent, "%")
```

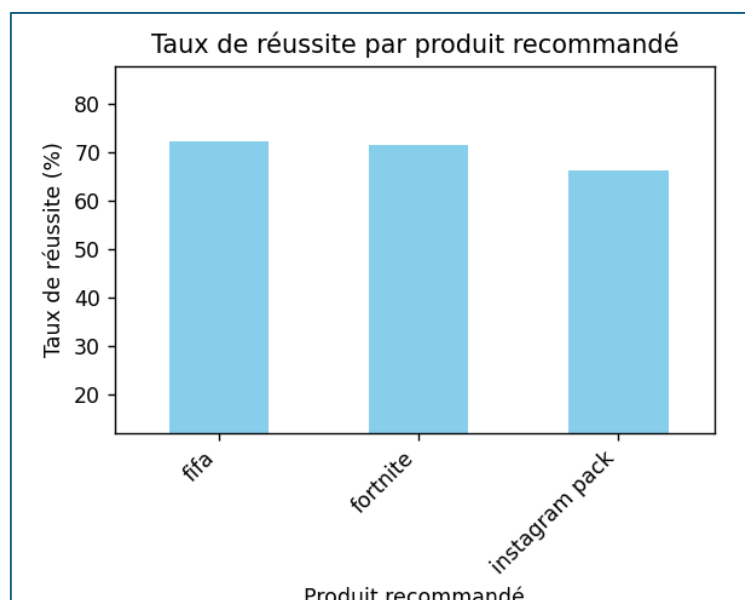
Le résultat est d'environ 69,74 %. Cela signifie que, dans ce jeu de données, près de 7 attaques sur 10 aboutissent, ce qui montre une population globalement assez vulnérable.

Ensuite, j'ai regardé le taux de réussite par produit recommandé (`recommended_product`). En prenant la moyenne de `campaign_success` pour chaque produit, j'obtiens :

- FIFA : environ 72,25 % de réussite,
- Fortnite : environ 70,22 %,
- Instagram pack : environ 66,22 %.

Même si les trois appâts sont efficaces (tous au-dessus de 65 %), les campagnes basées sur FIFA et Fortnite fonctionnent légèrement mieux que celles centrées sur le pack Instagram.

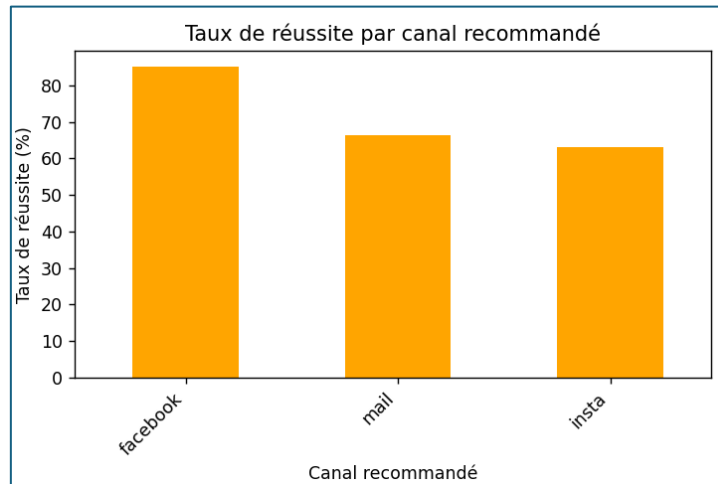
On peut en conclure que les thématiques liées aux jeux vidéo (et en particulier FIFA) sont un peu plus "cliquables".



J'ai fait la même chose pour le canal utilisé

- Facebook : environ 85,16 %,
- mail : environ 66,34 %,
- Instagram : environ 63,06 %.

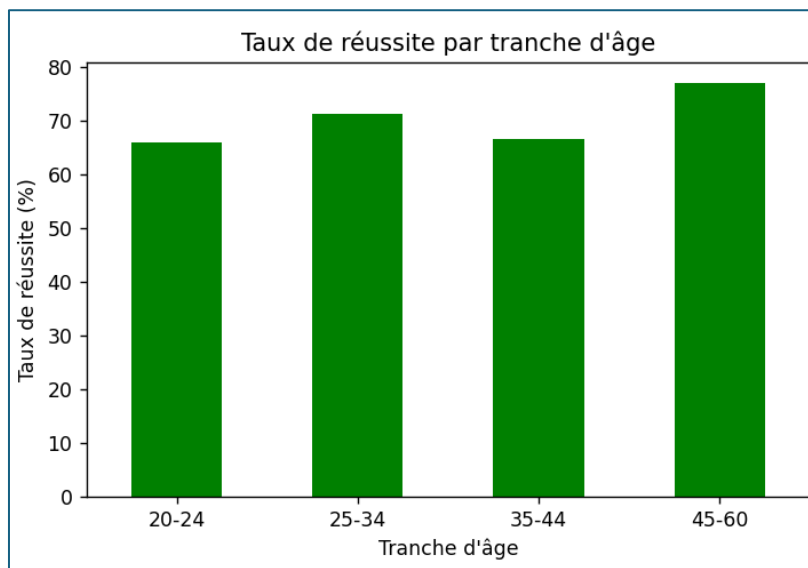
La différence est très nette : les attaques menées via Facebook réussissent beaucoup plus souvent que celles par mail ou Instagram. Pour une attaque, ce serait clairement le support à privilégier pour maximiser ses chances.



J'ai aussi étudié le taux de réussite par tranche d'âge, en créant des groupes (20–24, 25–34, 35–44, 45–60, 60+). Pour les personnes de 20 ans et plus, les résultats sont :

- 20–24 ans : 65,70 %,
- 25–34 ans : 70,33 %,
- 35–44 ans : 66,67 %,
- 45–60 ans : 76,92 %.

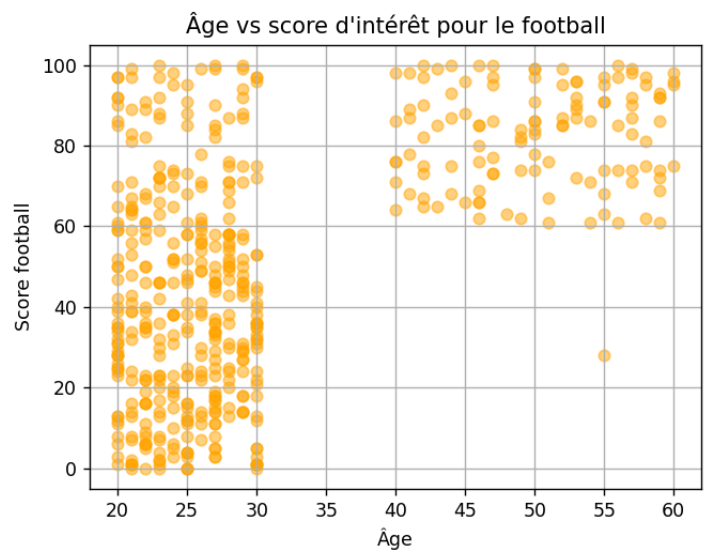
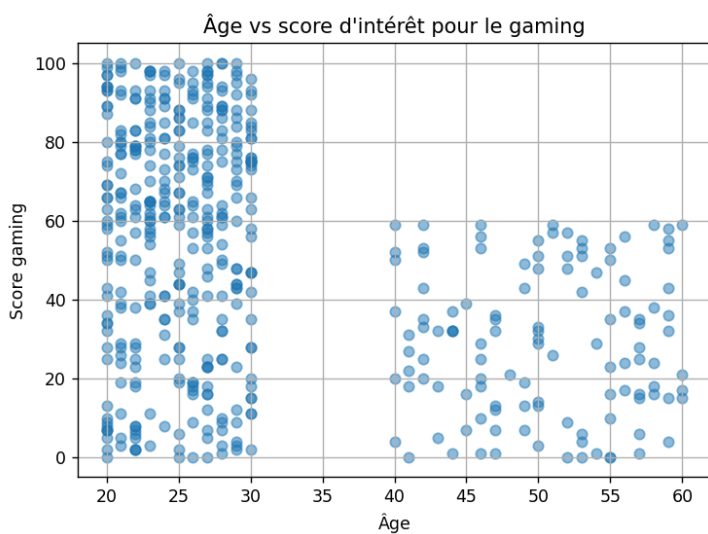
Toutes les tranches d'âge restent vulnérables, mais les 45–60 ans ont le taux de réussite le plus élevé, presque 77 %. Ce qui signifie que, dans ces données, les adultes de cette tranche d'âge sont ceux qui tombent le plus souvent dans le piège du phishing.



Enfin, pour aller un peu plus loin, j'ai essayé d'utiliser une matrice de corrélation entre les scores d'intérêt, l'âge et `campaign_success`. Au départ, je ne maîtrisais pas vraiment cette notion : je savais juste que la corrélation permettait de mesurer "le lien" entre deux variables (par exemple : est-ce que quand l'âge augmente, un score augmente aussi, diminue, ou ne change pas ?).

Mon objectif n'était pas de faire un modèle compliqué, mais simplement de voir s'il y avait des tendances visibles dans les données.

Pour rendre les choses plus visuelles, je les ai affichées sous forme de headmap, en traçant aussi deux nuages de points : un graphique âge vs score gaming et un graphique âge vs score football.



Sur le nuage de points "âge vs gaming", on voit clairement que les scores gaming sont plutôt élevés chez les plus jeunes et qu'ils diminuent avec l'âge. À l'inverse, sur le nuage "âge vs football", on observe que les profils plus âgés ont en général des scores de football plus élevés.

Ces deux graphiques confirment visuellement ce que montre la matrice de corrélation : dans ce dataset, les profils plus âgés sont plutôt "foot" et moins "gamer".

Datatelling : qui est la cible ?

À partir des KPI et des corrélations, on peut commencer à préciser le portrait d'un profil vulnérable dans ce jeu de données.

Globalement, près de 70 % des attaques réussissent, mais certains paramètres ressortent nettement : les produits liés au foot (FIFA), le canal Facebook et la tranche d'âge 45–60 ans.

Les statistiques montrent d'abord que les appâts basés sur FIFA ont un taux de réussite d'environ 72 %, légèrement supérieur à Fortnite (70 %) et à l'Instagram pack (66 %).

Les attaques menées via Facebook sont celles qui réussissent le plus souvent, avec un taux d'environ 85 %, bien plus élevé que les mails ou Instagram. Enfin, en regardant les tranches d'âge, les 45–60 ans sont le groupe le plus vulnérable, avec près de 77 % d'attaques réussies.

Les corrélations et les nuages de points complètent ce tableau : plus l'âge augmente, plus le score d'intérêt pour le football est élevé, tandis que le score d'intérêt pour le gaming diminue. Autrement dit, dans ce dataset, un profil âgé est rarement un “gamer”, mais il est souvent très intéressé par le foot.

En combinant ces éléments, un profil “type” se dégage : une personne de 45 à 60 ans, avec un score d'intérêt élevé pour le football, et qui utilise Facebook. D'un point de vue d'attaquant, c'est une cible intéressante, car les chiffres indiquent que ce segment réagit souvent positivement à des campagnes de phishing.

Scénario d'attaque

Sur la base de ce profil, on peut imaginer un scénario d'attaque cohérent avec les données :

La cible est un adulte d'environ 50 ans, très intéressé par le football, qui consulte régulièrement Facebook. Les statistiques montrent que les campagnes liées à des produits comme FIFA fonctionnent déjà bien et que Facebook est le canal le plus risqué. Un attaquant pourrait donc construire un message autour d'un thème très crédible pour ce profil : par exemple, un faux concours ou une offre spéciale liée au football.

Le message pourrait prendre la forme d'une publication sponsorisée ou d'un message privé sur Facebook, annonçant, par exemple, des billets de match à prix réduit, un accès exclusif à un pack FIFA “légendes”, ou une offre limitée de streaming. Le texte jouerait sur l'urgence (“offre valable seulement aujourd'hui”, “places limitées”) et sur la confiance (“partenaire officiel”, logos de clubs ou de marques de jeux connus).